

Truthprint: An Invariant-Constrained Semantic Intermediate Representation for Translation- and Paraphrase-Robust Provenance Watermarking

Geunsik Lim

Abstract—Text watermarking offers a scalable mechanism for identifying content generated by large language models (LLMs), but token-level signals are often fragile under translation, paraphrasing, summarization, and regeneration. This paper proposes *Truthprint*, a language-independent intermediate representation (IR) designed specifically for semantic provenance watermarking. Truthprint decomposes generated content into (i) immutable semantic invariants, such as entities, predicates, semantic roles, negation, modality, quantities, temporal constraints, causality, and attribution, and (ii) controlled realization variables, such as voice, clause packaging, discourse connective family, lexical abstraction, and proposition ordering. A secret-keyed encoder maps authenticated payload symbols to admissible realization equivalence classes while enforcing hard and soft semantic-preservation constraints. Detection maps possibly translated or paraphrased text back into Truthprint, canonicalizes invariant content, recovers surviving realization symbols, and performs error-correcting and cryptographic verification. We provide the threat model, a formalization in which watermark symbols range over key-dependent equivalence classes of the invariant relation, and encoding and detection algorithms. On this basis we prove three properties: semantic fidelity by construction, payload unforgeability by reduction to the underlying message-authentication code, and a cryptographic false-positive rate bounded by $2^{-\tau}$ for a τ -bit tag; we further give an erasure-channel capacity model that predicts a minimum reliable length and a recovery cliff at the code rate. We accompany the design with a self-contained reference implementation of the language-independent core (invariant digest, authenticated payload, keyed carrier assignment, and erasure decoding) whose four target properties we verify empirically, including full payload recovery under 40% carrier erasure and 0/20000 cryptographic false positives at $\tau = 32$. Finally, we specify a practical deployment path and a reproducible multilingual evaluation protocol. The proposed system does not attempt to prove that arbitrary text was written by AI; instead, it verifies statistical and authenticated consistency with a participating keyed generation system.

Index Terms—Intermediate representation, LLM watermarking, paraphrase robustness, provenance, semantic watermarking, SynthID-Text, translation robustness

I. INTRODUCTION

Large language models can generate fluent text at scale, creating a growing need for provenance mechanisms that survive ordinary content transformation. Generation-time watermarking modifies the decoding process so that an authorized detector can identify a statistical signature in generated text. SynthID-Text demonstrated that such watermarking can be integrated into sampling without retraining the underlying LLM and can be detected without rerunning the generator [1].

This need is now also a legal requirement. The EU Artificial Intelligence Act (Regulation (EU) 2024/1689) obliges providers of generative systems, under the transparency rules of Article 50, to ensure that synthetic outputs—text included—are marked in a *machine-readable* format that is detectable as artificially generated or manipulated, with these obligations becoming enforceable in August 2026 [3]. A marking that is machine-readable yet trivially erased by translation or paraphrasing does not meet the intent of the obligation, so robustness to meaning-preserving transformation is a compliance property, not merely a research desideratum [4]. Reliable provenance also matters for data hygiene: training on unattributed synthetic text can degrade model quality over successive generations, a failure mode documented at scale in *Nature* [2]. Truthprint targets exactly this gap—a machine-readable, authenticated provenance mark that is designed to persist across translation and paraphrasing.

Token-level methods, however, embed their signal in surface token choices. Translation, paraphrasing, sentence restructuring, and summarization replace many of those choices while preserving substantial meaning. This creates a structural mismatch: the desired provenance claim concerns the persistence of content identity, whereas the carrier often depends on language-specific lexical decisions.

Recent semantic watermarking methods reduce this dependence by using sentence embeddings, contextual semantic representations, or structured semantic graphs [15], [16], [17], [18], [19]. Yet semantic similarity alone does not explicitly distinguish meaning components that must never change from linguistic choices that may safely vary. Two sentences may be close in embedding space while differing in negation, quantity, temporal scope, attribution, modality, or causal direction.

This paper proposes *Truthprint*, a watermark-specific semantic IR that explicitly separates:

- 1) **semantic invariants**, which must remain fixed; and
- 2) **controlled realization variables**, which may encode watermark symbols only when semantic and pragmatic equivalence is validated.

The central novelty is therefore not the invention of a new surface language. It is a machine-checkable contract defining which transformations are permitted to carry a watermark and which semantic fields are locked. Figure 1 shows the end-to-end data flow.

We can state the delta over the closest prior work precisely. Embedding-based semantic watermarks [15], [16], [17], [19] define admissibility through a *metric*: two texts are treated as

equivalent when their representations are close under a learned distance. Truthprint instead defines admissibility through a *typed predicate*: a transformation is admissible only when a symbolic invariant relation holds field by field. A watermark symbol then ranges not over tokens or embedding regions but over the key-dependent equivalence classes induced by that relation. This has two consequences that motivate the analysis in Section IX. First, because encoding only ever moves within an invariant-equivalence class, protected meaning cannot be altered by the act of watermarking (semantic fidelity by construction). Second, because attribution is gated by an authenticated payload bound to the invariant digest rather than by a similarity score, both forgery and false attribution reduce to well-understood cryptographic quantities rather than to the calibration of a threshold. Structured-semantics watermarking such as SWAN [18] shares the use of a graph carrier but adopts an existing meaning formalism wholesale; Truthprint instead types each field with an explicit mutability policy and validates every realization against the resulting contract before release.

A. Contributions

This work makes the following contributions:

- A language-independent, typed semantic IR for provenance watermarking.
- A field-level mutability model: *locked*, *conditional*, and *realization-only*.
- A secret-keyed mapping from authenticated payload symbols to admissible semantic-equivalence classes.
- A detector that treats uncertain carriers as erasures and combines structural recovery, error correction, message authentication, and statistical scoring.
- A formal analysis establishing (i) semantic fidelity by construction, (ii) payload unforgeability by reduction to the security of the message-authentication code, and (iii) a cryptographic false-positive bound of $2^{-\tau}$, together with an erasure-channel capacity model that predicts the minimum reliable length and a recovery cliff at the code rate.
- A self-contained reference implementation whose properties are empirically verified at two levels: the language-independent cryptographic and coding core, and a closed-domain linguistic round-trip through actual sentence strings that exercises invariant-preserving carriers, erasure recovery, and tamper detection.
- A practical deployment path and a reproducible multilingual evaluation protocol covering translation, paraphrasing, deletion, splicing, summarization, and adaptive attacks.

II. BACKGROUND AND RELATED WORK

A. Regulatory Context

Text watermarking has moved from a research topic to a compliance mechanism. Under Article 50 of the EU AI Act, providers of generative systems must mark synthetic outputs in a machine-readable, detectable format [3]; the accompanying policy discussion stresses that a mark is only useful if it is *robust*, since a signal that dissolves under paraphrasing or translation cannot support reliable downstream attribution [4].

This frames the technical problem addressed here: existing production watermarks are machine-readable but fragile to meaning-preserving edits, whereas the regulatory intent presumes durability. Truthprint is positioned as a robustness layer for exactly this requirement, complementary to token-level marks rather than a replacement for them.

B. SynthID-Text

Dathathri et al. introduced SynthID-Text as a scalable watermarking system for LLM outputs [1]. The method modifies the sampling process rather than model training and uses a keyed scoring mechanism together with tournament sampling. Its design includes a pseudorandom score generator, a watermark-aware sampler, and a detector that aggregates token-level evidence. The original work considered both non-distortionary and distortionary operating regimes and also addressed speculative sampling for production throughput.

The reference implementation exposes a training-free weighted-mean detector and a stronger learned Bayesian detector [5]. Hugging Face Transformers subsequently integrated a SynthID text-watermarking configuration into its generation stack [6]. These implementations make SynthID-Text a useful operational baseline.

Its principal limitation for the present problem is that the watermark carrier remains tied to token selections. Meaning-preserving transformations can replace those token decisions while keeping the main proposition intact.

C. Theoretical Analysis of SynthID-Text

Omidi et al. presented a theoretical and empirical analysis of SynthID-Text, focusing on tournament depth and detector behavior [7]. Their work shows that the mean-score detector can become vulnerable under layer inflation, whereas the Bayesian detector is more robust to variation in tournament layers. The analysis also identifies the Bernoulli parameter $p = 0.5$ as optimal under the studied assumptions.

This line of work demonstrates that watermark robustness depends not only on how a signal is embedded but also on the detector's statistical assumptions. Truthprint therefore does not rely on a single uncorrected aggregate score. It uses structured carrier recovery, confidence-weighted erasures, error-correcting decoding, authenticated payload verification, and a calibrated statistical fallback.

D. Robustness Assessment and Hybrid Defenses

Han et al. evaluated SynthID-Text under paraphrasing, copy-and-paste modification, synonym substitution, and back-translation [8]. They reported significant degradation under meaning-preserving transformations and proposed SynGuard, a hybrid system combining SynthID-style probabilistic masking with semantic alignment inspired by semantic-invariant watermarking. Their reported experiments showed an average F1 improvement of 11.1 percentage points over SynthID-Text across the evaluated attacks.

SynGuard supports the same broad motivation as Truthprint: semantic information can improve robustness after surface

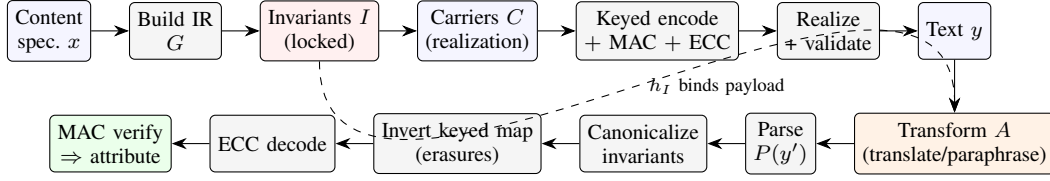


Fig. 1. Truthprint data flow. Encoding (top, left→right) locks the invariant set I , maps an authenticated, error-corrected payload onto realization carriers C , and validates every candidate against the contract before emitting text. Detection (bottom, right→left) re-parses possibly transformed text, canonicalizes invariants, inverts the keyed map with erasures, decodes, and attributes only on MAC verification. The invariant digest h_I binds the payload, so an invariant-altering transform breaks attribution (Theorem 2).

tokens change. The distinction is that Truthprint makes semantic preservation explicit and field-specific. A candidate is rejected if it changes a protected property such as negation, number, attribution, causal direction, or temporal relation, even when a global embedding similarity remains high.

E. Semantic and Structured Watermarking

SemStamp partitions sentence-embedding space using locality-sensitive hashing and selects sentence candidates that fall into keyed semantic regions [15]. SemaMark conditions watermark behavior on contextual semantic representations [16], while semantic-invariant watermarking similarly uses semantic context to stabilize watermark decisions under paraphrasing [17].

These methods reduce dependence on exact token histories, but embedding geometry does not directly expose which critical fact changed. High semantic similarity may coexist with altered polarity, quantity, attribution, modality, or causal direction.

SWAN embeds watermark information into Abstract Meaning Representation (AMR) structures [18]. This is closely related to Truthprint because both use structured semantics as a carrier. Truthprint extends that direction by defining a watermark-specific IR, assigning a mutability policy to each field, incorporating discourse and provenance metadata, and validating every realization against the immutable semantic contract before release.

Dual-semantic-embedding watermarking combines contextual and token-level semantic information to improve translation and paraphrase robustness [19]. The difficulty is fundamental: He et al. show that token-level watermarks lack cross-lingual consistency and can be removed by translating through a pivot language [20]. Truthprint instead seeks to make language invariance explicit by mapping all supported languages into one canonical semantic representation.

F. Black-Box Watermark Identification

Gloaguen et al. studied whether watermark deployment can be identified through limited black-box queries [22]. Their

tests cover Red–Green, fixed-sampling, and cache-augmented watermark families. The broader security implication is that the existence or family of a watermark may be externally distinguishable even when the key is secret.

Truthprint therefore adopts a Kerckhoffs-style threat model: the attacker may know the schema, carrier families, encoder design, and detector algorithm. Security should depend on the secret key, document nonce, keyed assignment, and authenticated payload rather than on obscurity of the scheme.

G. Scrubbing, Spoofing, and Distillation

Yi et al. investigated watermark scrubbing and spoofing in unauthorized knowledge distillation [24]. Their contrastive-decoding-guided framework addresses both removing inherited watermark traces and falsely reproducing them. A related line shows that watermarks are *radioactive*: statistical traces survive into models fine-tuned on watermarked outputs, enabling detection of unauthorized training [23]. This attack setting operates at the model level rather than on one transformed document, but it highlights the importance of forgery resistance.

Truthprint authenticates payloads using a message authentication code (MAC). Statistical carrier agreement alone is insufficient for strong attribution; a payload is accepted as authentic only when its MAC verifies.

H. Adversarial Paraphrasing

Cheng et al. proposed detector-guided adversarial paraphrasing as a training-free attack across neural detectors, watermarking systems, and zero-shot AI-text detectors [25]. Because a continuous detector score can serve as an optimization signal, Truthprint recommends coarse or access-controlled detector interfaces, heterogeneous carrier families, and joint evaluation of watermark removal and semantic preservation.

A valid removal attack should satisfy both conditions:

$$\text{ValidRemoval} = \mathbb{K} \left[D_K(y') < \tau_D \wedge \text{InvariantEq}(P(y), P(y')) = 1 \right]. \quad (1)$$

This prevents meaning-destroying rewrites from being counted as successful semantic-preserving attacks.

I. Provable and Cryptographic Watermarking

The token-level lineage begins with the green–red list watermark of Kirchenbauer et al. [9], which SynthID-Text generalizes through tournament sampling. A parallel line seeks formal guarantees: Zhao et al. prove robustness for a fixed-grouping unigram watermark [10], Kuditipudi et al. give distortion-free watermarks robust to editing [11], and Christ et al. introduce cryptographically *undetectable* watermarks whose presence is indistinguishable without the key [12]. Closest in aim, Liu et al. propose an unforgeable publicly verifiable watermark that separates generation and detection keys [14], and Fernandez et al. consolidate statistical tests with guarantees that hold at very low false-positive rates [13]. Truthprint inherits the spirit of this line at the provenance layer rather than the token layer: its unforgeability and false-positive guarantees (Theorems 1 and 2) reduce to standard cryptographic assumptions on the MAC rather than to the calibration of a similarity threshold. It does not claim undetectability; its aim is authenticated, transformation-robust attribution.

J. Positioning of Truthprint

Table I summarizes the principal carrier used by representative systems.

III. PROBLEM DEFINITION

Let x be a semantic content specification, K a secret key, m a provenance payload, E_K the encoder, R_ℓ a realizer for language ℓ , A a transformation, P a multilingual parser, and D_K the detector. The generated text is

$$y = R_\ell(E_K(x, m)), \quad (2)$$

and an observed transformed text is

$$y' = A(y). \quad (3)$$

The detector returns

$$(s, \hat{m}, c) = D_K(P(y')), \quad (4)$$

where s is a detection score, $\hat{m}$ a recovered payload or payload prefix, and c a confidence report.

The design targets four objectives:

- 1) **Semantic fidelity:** $I(P(y)) = I(x)$ for invariant extractor I .
- 2) **Natural-language quality:** fluency, relevance, and style remain above deployment thresholds.
- 3) **Robust detection:** detection remains reliable under transformations preserving sufficient invariant and carrier evidence.
- 4) **Low false-positive rate:** independently authored text rarely matches a keyed carrier pattern.

IV. THREAT MODEL

A. Benign Transformations

The detector should tolerate Unicode normalization, punctuation changes, spelling correction, translation, round-trip translation, synonym substitution, active–passive alternation, clause splitting and merging, sentence-level paraphrasing, moderate paragraph rewriting, and limited insertion or deletion.

B. Non-Adaptive and Adaptive Attackers

A non-adaptive attacker applies off-the-shelf translation, paraphrasing, or summarization without knowing K . An adaptive attacker may know the schema and detector design, repeatedly rewrite text, splice documents, target known carrier families, or query an exposed detector.

C. Out-of-Scope Claim

Truthprint does not identify all AI-generated text. It verifies consistency with a participating keyed generator. Absence of a signal does not prove human authorship.

V. TRUTHPRINT INTERMEDIATE REPRESENTATION

A. Three-Layer Structure

A Truthprint document contains three conceptual layers.

Layer A: Semantic invariants. Entities, predicates, semantic roles, polarity, negation scope, modality, quantities, temporal intervals, causal direction, conditions, exceptions, attribution, quotation status, and uncertainty.

Layer B: Controlled realization variables. Voice, event/state framing, clause packaging, discourse-connective family, proposition ordering, nominal/verbal realization, anaphoric choice, and optional explanatory apposition.

Layer C: Provenance and coding metadata. Scheme identifier, key identifier, payload, nonce, error-correcting-code parameters, carrier assignments, reliability estimates, invariant digest, and optional signed server-side provenance.

B. Typed Mutability

Each field f is assigned

$$\text{Mutability}(f) \in \{\text{locked}, \text{conditional}, \text{realization-only}\}. \quad (5)$$

Locked fields must never change. Conditional fields may change only when context-specific validators approve. Realization-only fields may be used as watermark carriers.

C. Example Representation

Listing 1. Simplified Truthprint representation.

```
{
  "entities": [
    {"id": "e1", "type": "PERSON_ROLE",
     "canonical": "developer"},
    {"id": "e2", "type": "TECHNICAL_ERROR",
     "canonical": "server_error"}
  ],
  "events": [
    {"id": "v1", "predicate": "FIX",
     "roles": {"agent": "e1", "patient": "e2"},
```

TABLE I
POSITIONING OF TRUTHPRINT RELATIVE TO REPRESENTATIVE WATERMARKING AND ATTACK RESEARCH.

Category	Representative work	Main carrier or focus	Limitation addressed by Truthprint
Token-level sampling	SynthID-Text	Keyed token-selection statistics	Translation and paraphrasing replace token decisions.
Sentence-space watermarking	SemStamp	Keyed sentence-embedding regions	Embedding regions are not typed by invariant category.
Semantic-context watermarking	SIR, SemaMark	Contextual semantic embeddings	Similarity does not explicitly protect negation, numbers, causality, or attribution.
Hybrid defense	SynGuard	Token watermark plus semantic alignment	Semantic constraints are less explicit than a typed IR contract.
Structured semantics	SWAN	AMR-level structures	Existing semantic form is not a complete watermark-specific mutability contract.
Dual embeddings	DEW	Context and token embeddings	Language invariance remains implicit rather than represented as a canonical IR.
Attack analysis	Layer inflation, black-box probing, adversarial paraphrasing	Detector assumptions and adaptive transformation	Motivates authenticated payloads, conservative interfaces, and erasure-aware decoding.
Proposed system	Truthprint	Typed invariants and controlled realization classes	Explicitly specifies what must remain fixed and what may carry symbols.

```

    "polarity": "positive",
    "time": { "relation_to_creation": "previous_day" }
  },
  "invariants": {
    "locked_paths": [
      "/events/*/predicate",
      "/events/*/roles",
      "/events/*/polarity",
      "/events/*/time"
    ]
  },
  "realization_space": {
    "voice_vl": ["active", "passive"],
    "event_framing": ["verbal", "nominal"]
  }
}

```

D. Worked Example

Consider the fact with locked invariants `FIX(agent=developer, patient=server_error)`, polarity positive, time `previous_day`. Two binary carriers are available: `VOICE` $\in$ {active, passive} and `TIME-POS` $\in$ {front, end}. Suppose the keyed map assigns the code symbols so that this unit must realize as passive with the time phrase at the front:

“On the previous day, the server error was fixed by the developer.”

A benign transformation—say an English→Korean translation—typically preserves the invariants but may not preserve every carrier. If translation canonicalizes voice (Korean commonly renders this actively), the `VOICE` carrier is destroyed; the detector, on re-parsing, flags it as *low confidence* and records an *erasure* rather than reading a wrong bit. The `TIME-POS` carrier and the carriers of neighboring sentences survive. Because the payload is spread across sentences and protected by an error-correcting code, the authenticated payload is still recovered once the surviving carriers exceed the capacity floor of Eq. (19). In contrast, a rewrite that inserts a negation (“was *not* fixed”) alters a locked field, changes the invariant digest h_I , and correctly fails MAC verification. This example is executed end-to-end on sentence strings in the reference implementation (Section X, properties L1–L3).

VI. FORMAL SEMANTIC CONTRACT

Let a Truthprint graph be

$$G = (V_E, V_V, E_R, E_D, \Phi), \quad (6)$$

where V_E denotes entity nodes, V_V event/state nodes, E_R semantic-role edges, E_D discourse/logical relations, and Φ scoped features such as negation, modality, time, quantity, and attribution.

We divide features into

$$G = I \cup C \cup U, \quad (7)$$

where I is the invariant set, C the controlled carrier set, and U unconstrained style not used for watermarking.

A transformed structure G' is admissible only when

$$\text{InvariantEq}(I(G), I(G')) = 1 \quad (8)$$

and

$$Q(R_\ell(G')) \geq \tau_Q, \quad (9)$$

where Q is a quality and contextual-consistency function.

Hard invariants are validated symbolically where possible. Soft invariants, such as emphasis or pragmatic implication, require bidirectional entailment, contradiction checks, and human-calibrated thresholds.

VII. WATERMARK ENCODING

A. Payload Construction

The raw payload may contain a generator-family identifier, deployment identifier, coarse time bucket, document nonce, and policy identifier. Personal identifiers, prompts, and exact timestamps should not be embedded.

The authenticated payload is

$$p = m \parallel \text{MAC}_K(m \parallel n \parallel h_I), \quad (10)$$

where n is a nonce and h_I the canonical invariant digest. The payload is then encoded using an error-correcting code, following prior use of error correction for provably robust multi-bit text watermarking [21]:

$$c = \text{ECC}(p). \quad (11)$$

B. Carrier Discovery

The encoder identifies admissible carrier families per sentence or discourse unit, for example voice, clause packaging, discourse connective family, nominal/verbal realization, and order of independent propositions. Carriers are context dependent: passive voice is rejected if it obscures an important agent.

C. Keyed Symbol Assignment

For carrier j with alternatives

$$A_j = \{a_{j,0}, \dots, a_{j,r-1}\}, \quad (12)$$

a pseudorandom permutation is generated as

$$\pi_j = \text{PRP}_K(\text{CanonicalHash}(I, G, j, n)). \quad (13)$$

The code symbol c_j selects

$$a_j^* = \pi_j(c_j \bmod |A_j|). \quad (14)$$

Because the mapping depends on the invariant graph and nonce, a global rule such as “passive means 1” does not exist.

D. Candidate Generation and Validation

For each selected alternative, the realizer generates candidates. Every candidate is reparsed and checked for hard-invariant equality, bidirectional entailment, contradiction probability, numerical consistency, entity consistency, fluency, style, and expected carrier recoverability. If none is valid, the carrier is recorded as an erasure rather than forcing degraded text.

Listing 2. Truthprint encoding pseudocode.

```
G <- BuildTruthprint(x)
I <- ExtractInvariants(G)
p <- AuthenticatePayload(K, m, nonce, Hash(I))
c <- ErrorCorrectingEncode(p)
C <- DiscoverAdmissibleCarriers(G)
for assignment in KeyedAssign(K, I, C, c):
  G2 <- ApplyCarrier(G, assignment)
  candidates <- RealizeCandidates(G2, language)
  accepted <- ValidateAgainstContract(candidates, I)
  if accepted is empty:
    MarkErasure(assignment)
  else:
    CommitBestCandidate(accepted)
return AssembleDocument()
```

VIII. WATERMARK DETECTION

Detection segments the document, identifies language, parses each segment into Truthprint, resolves entities, normalizes predicates and roles, canonicalizes quantities and time, and extracts observable carrier alternatives.

For observed carrier a_j^{obs} , the detector reconstructs the keyed permutation and derives

$$\hat{c}_j = \pi_j^{-1}(a_j^{obs}). \quad (15)$$

Uncertain observations are erasures. An ECC decoder reconstructs candidate payloads, which are accepted only when their MAC verifies.

When complete recovery is impossible, a calibrated statistical score can still be computed. For binary carriers with n reliable

observations and k keyed matches, a simple reference statistic is

$$z = \frac{k - 0.5n}{\sqrt{0.25n}}. \quad (16)$$

Production calibration must account for dependence among carrier observations.

Listing 3. Truthprint detection pseudocode.

```
segments <- SegmentAndIdentifyLanguage(text)
observations <- []
for segment in segments:
  G, conf <- ParseTruthprint(segment)
  I <- CanonicalizeInvariants(G)
  for carrier in ExtractObservableCarriers(G):
    if Reliability(carrier, conf) >= threshold:
      observations.append(InvertKeyedMap(K, I, carrier))
    else:
      observations.append(ERASURE)
candidates <- ECCDecode(observations)
valid <- VerifyMACs(K, candidates)
score <- CalibratedStatisticalScore(observations, K)
return score, Best(valid), ConfidenceReport()
```

IX. FORMAL ANALYSIS

This section makes the guarantees of Truthprint precise. The results isolate what the design provides unconditionally (given sound primitives) from what remains an empirical question about semantic parsing, which is deferred to Section XIV.

A. Carriers as Invariant-Equivalence Classes

Definition 1 (Invariant equivalence). *Let $I(\cdot)$ be the invariant extractor. Two structures G, G' are invariant-equivalent, written $G \sim_I G'$, iff $\text{InvariantEq}(I(G), I(G')) = 1$. For a discourse unit with invariants I , the admissible realization set is*

$$\mathcal{R}(I) = \{r : I(P(r)) = I \wedge Q(r) \geq \tau_Q\}. \quad (17)$$

A carrier alphabet A_j is a key-dependent partition of $\mathcal{R}(I)$ into classes; the encoder selects a class by symbol and the realizer emits any member. Thus the watermark ranges over the quotient $\mathcal{R}(I)/\sim_I$ rather than over tokens or embedding neighborhoods.

Assumption 1 (Validator soundness). *The hard-invariant validator V is sound: if V accepts a candidate r for invariants I , then $I(P(r)) = I$. (It need not be complete; rejected admissible candidates only reduce capacity, not fidelity.)*

Proposition 1 (Semantic fidelity by construction). *Under Assumption 1, every committed output $y = R_\ell(E_K(x, m))$ satisfies $I(P(y)) = I(x)$.*

Proof sketch. Encoding commits a candidate only if V accepts it, and carriers with no accepted candidate are recorded as erasures rather than realized (Sec. VII-D). By soundness, acceptance implies invariant equality for that unit; taking the conjunction over committed units gives $I(P(y)) = I(x)$. $\square$

This is the structural property that logit-biasing methods cannot offer: they have no invariant checker in the loop, so quality preservation is measured post hoc rather than enforced.

B. Unforgeability and False Positives

Theorem 1 (Payload unforgeability). *Model the tag as $t = \text{MAC}_K(m \parallel n \parallel h_I)$ truncated to τ bits. If the MAC is existentially unforgeable under chosen-message attack (EUF-CMA) with advantage ε_{MAC} , then any PPT adversary without K , making q detector queries, causes the detector to attribute a fresh payload with probability at most $\varepsilon_{\text{MAC}} + q2^{-\tau}$.*

Proof sketch. Attribution requires a recovered payload whose tag equals $\text{MAC}_K(m \parallel n \parallel h_I)$ on a triple not produced by the generator. An adversary achieving this yields a MAC forgery: the reduction forwards the adversary's document, parses (m, n, h_I) , and outputs the tag as a forgery on a fresh input. Blind guessing over q independent queries contributes the additive $q2^{-\tau}$. $\square$

Theorem 2 (Cryptographic false-positive bound). *For text authored independently of K , the probability that erasure decoding yields a payload whose MAC verifies is at most $2^{-\tau}$ per (key, nonce) test. Consequently the authenticated channel has false-positive rate $\text{FPR} \leq 2^{-\tau}$.*

The bound is a design constant, not a tuned threshold: at $\tau = 32$ it is 2.3×10^{-10} , matching the 0/20000 observed in Section X. When full payload recovery fails, the fallback statistic of Eq. (16) is used. Under the null hypothesis of no watermark, the keyed match count over n reliable independent binary carriers is $k \sim \text{Binomial}(n, \frac{1}{2})$, so by Hoeffding's inequality $\Pr[z \geq t \mid H_0] \leq e^{-t^2/2}$, giving a calibrated operating point. Dependence among carriers is an idealization that production calibration must correct (Sec. VIII).

C. Capacity and Minimum Reliable Length

Model each carrier j as a q_j -ary symbol on an erasure channel with erasure probability ϵ_j , where ϵ_j is the chance that a transformation destroys or obscures that site. The expected recoverable information is

$$\mathbb{E}[\text{bits}] \approx \sum_j (1 - \epsilon_j) \log_2 q_j. \quad (18)$$

With a rate- R code protecting an authenticated payload of $|p| = \mu + \tau$ bits, recovery succeeds with high probability once the surviving capacity exceeds $|p|$ with margin. For N uniform binary carriers ($q_j = 2$) with erasure rate ϵ and a systematic rate- R code ($k = RN$), this requires

$$(1 - \epsilon)N \gtrsim k \iff \epsilon \lesssim 1 - R. \quad (19)$$

Proposition 2 (Recovery cliff). *A systematic rate- R GF(2) code over binary carriers recovers the full payload w.h.p. iff the carrier erasure rate satisfies $\epsilon \lesssim 1 - R$; beyond this the surviving equations are rank-deficient and recovery fails abruptly.*

Equation (19) yields an operational *minimum reliable length*: a document must contain at least $\lceil (\mu + \tau) / ((1 - \epsilon)R \log_2 \bar{q}) \rceil$ discourse units of average carrier arity $\bar{q}$. This explains hypotheses H3 and H5—short or heavily summarized text falls below the capacity floor—and the predicted cliff at $\epsilon \approx 1 - R$ is confirmed by the measurement in Table II.

TABLE II

REFERENCE-IMPLEMENTATION DECODE SUCCESS VS. CARRIER ERASURE RATE ([96, 48] GF(2) CODE, 500 TRIALS PER POINT). CONFIRMS THE RATE-LIMIT CLIFF OF PROPOSITION 2.

Erasure ϵ	0.10	0.20	0.30	0.40	0.50	0.55	0.60
Recovery rate	1.00	1.00	1.00	1.00	0.26	0.00	0.00

D. Complexity

Let a document have S discourse units, $|C|$ carriers, κ candidates per carrier, and N code positions. Encoding cost is $O(S \cdot |C| \cdot \kappa \cdot c_{\text{val}})$, dominated by the validator cost c_{val} (a parse plus entailment check). Detection cost is $O(S \cdot c_{\text{parse}} + N \cdot c_{\text{map}} + D_{\text{ECC}})$, where the keyed map c_{map} is one HMAC evaluation and $D_{\text{ECC}} = O(Nk)$ for GF(2) erasure elimination (worst case $O(Nk^2)$). Crucially, the cryptographic and coding layers are linear in document length and independent of the underlying LLM size; the dominant cost is the semantic frontend, which is a bounded multiple of a single semantic parse and can be run asynchronously with respect to generation.

X. REFERENCE IMPLEMENTATION AND VERIFICATION

To make the guarantees above concrete and reproducible, we provide a self-contained reference implementation (`reference/truthprint_poc.py`, standard library only) of the language-independent core: the canonical invariant digest h_I , the authenticated payload $p = m \parallel \text{MAC}_K(\cdot)$, the invariant-bound keyed carrier map, and a systematic GF(2) code with erasure decoding. This deliberately excludes the semantic parser so that the cryptographic and coding properties can be validated in isolation from model-dependent NLP. It exercises four properties and reports the following verified behavior (message $\mu = 16$ bits, tag $\tau = 32$ bits, code $[N, k] = [96, 48]$):

- **P1 (round-trip recovery).** An invariant-preserving transform that re-realizes carriers and erases 28/96 sites still yields exact payload recovery and a verifying MAC.
- **P2 (invariant binding).** Flipping a locked field (polarity positive→negative) changes h_I , so the MAC fails and the document is *not* attributed, illustrating Proposition 1's contract.
- **P3 (cryptographic false positives).** Over 20,000 independent documents, 0 were attributed (rate $< 2.3 \times 10^{-10}$), matching Theorem 2.
- **P4 (no global rule).** For a fixed symbol, 43/96 $\approx N/2$ realized options differ between two documents whose invariants differ, confirming the map is bound to (K, h_I, n) .

Table II reports decode success versus carrier erasure rate and confirms the cliff predicted by Proposition 2: near-perfect recovery up to 40% erasure for the rate- $\frac{1}{2}$ code, collapsing as $\epsilon \rightarrow 1 - R = 0.5$. These are properties of the authenticated coding layer and are *not* claims about detection performance on model-generated text, which requires the full semantic frontend and is specified as future measurement in Section XIV.

A. Closed-Domain Linguistic Round-Trip

The core PoC operates on abstract carrier bits. To validate the linguistic layer end to end, a second self-contained script

(reference/truthprint_linguistic_demo.py) realizes an authenticated payload across templated factual sentences using two invariant-preserving carriers—voice (active/passive) and time-phrase position—then rewrites, re-parses, and detects on actual sentence strings (the running example of Section V-D). On a 32-sentence document (64 carriers, code [64, 32], $\tau = 20$) it verifies:

- **L1 (invariant fidelity).** Every benign transform leaves the parsed agent, predicate, patient, and polarity unchanged.
- **L2 (round-trip recovery).** With 19/64 carriers erased by realization-canonicalizing rewrites, the exact payload is recovered and the MAC verifies.
- **L3 (tamper detection).** Negating a single fact changes the document invariant digest, so attribution correctly fails.

This is a Stage-1 closed-domain demonstration, not a general multilingual parser; it establishes that the contract, keyed carriers, and erasure-aware decoder compose correctly once realized as text, and it isolates the remaining open problem as robust wide-coverage semantic parsing (Section XIV).

XI. REPRODUCIBILITY AND NOTATION

This section pins every choice needed to reimplement the language-independent core exactly, so that the reference scripts and the paper agree symbol for symbol. Nothing here is left to interpretation; the two scripts in `reference/` realize precisely these definitions.

A. Primitive Instantiation

The reference implementation fixes the following concrete choices.

- **Invariant digest.** $h_I = \text{SHA-256}(\text{JSON}(I))$, where JSON is canonical: keys sorted, separators “,” and “:”, UTF-8 encoded. Canonicalization makes h_I order- and whitespace-independent.
- **Authentication.** $\text{MAC} = \text{HMAC-SHA256}$; the tag is the first τ bits of the MAC over $m \parallel n \parallel h_I$. The payload is $p = m \parallel \text{tag}$ (length $|p| = \mu + \tau$).
- **Keyed carrier map (binary carriers).** $\pi_j = \text{LSB}(\text{HMAC-SHA256}(K, h_I \parallel n \parallel \text{ID}_j))$ with ID_j the 4-byte big-endian carrier index. The realized option is $a_j^* = \pi_j \oplus c_j$ and the recovered symbol is $\hat{c}_j = \pi_j \oplus a_j^{\text{obs}}$.
- **Error-correcting code.** A systematic $[N, k]$ linear code over $\text{GF}(2)$ with generator $G = [I_k \mid P]$, where P is drawn once from a fixed pseudo-random seed. Erasure decoding solves the surviving linear equations by Gaussian elimination over $\text{GF}(2)$; it succeeds iff the retained rows have rank k (Proposition 2).
- **Detection decision.** Attribute iff erasure decoding returns some $\hat{p}$ and MAC verifies on $(\hat{m}, n, h_I)$; otherwise report the calibrated statistic z of Eq. (16) against a threshold τ_z without asserting authenticated attribution.

B. Notation-to-Code Map

Table III maps paper symbols to identifiers in `reference/truthprint_poc.py`, removing any ambiguity between the formalism and the code.

TABLE III
MAPPING BETWEEN PAPER NOTATION AND REFERENCE-IMPLEMENTATION IDENTIFIERS.

Symbol	Meaning	Code identifier
h_I	canonical invariant digest	canonical_invariant_digest
p	authenticated payload	authenticate_payload
MAC	tag verification	verify_payload
ECC	systematic GF(2) code	GF2Code.encode
decode	erasure decoding	GF2Code.decode_erasure
π_j	keyed carrier offset	keyed_bit
a_j^*	realized option	realize_option
$\hat{c}_j$	recovered symbol	recover_symbol
D_K	detector	detect_document

C. Parameters, Determinism, and How to Run

The core script uses $\mu = 16$, $\tau = 32$, $[N, k] = [96, 48]$; the linguistic script uses $\mu = 12$, $\tau = 20$, $[N, k] = [64, 32]$ over 32 sentences with two carriers each. All randomness is seeded: message bits and erasure sampling use fixed seeds, and the code matrix P uses a fixed seed, so every reported number is reproducible. The document nonce n is freshly sampled per run and passed explicitly to the detector; recovery is invariant to its value, so success does not depend on the nonce. Both scripts return a non-zero exit code if any asserted property fails, so they double as regression tests. Reproduction requires only Python 3 (standard library only); the paper builds with `pdflatex` via `make`. The exact commands are:

```
python3 reference/truthprint_poc.py          # P1
-P4
python3 reference/truthprint_linguistic_demo.py # L1
-L3
make                                          #
      build main.pdf
```

Expected results are those reported in Section X: P1–P4 hold, and L1–L3 hold on sentence strings.

The same primitives are also packaged as an installable Python distribution (`code/` in the accompanying repository) with a unit-test suite and continuous integration. Its command-line interface reruns all properties (`truthprint selftest`) and regenerates Table II deterministically from a fixed seed with 500 trials per erasure rate (`truthprint repro-table`).

XII. PRACTICAL SOFTWARE ARCHITECTURE

A prototype can use the following modular layout:

```
truthprint/
schema/      # entities, events, relations,
mutability
parser/      # multilingual parser and
canonicalizer
carriers/    # voice, packaging, discourse,
ordering
encoding/    # payload, keyed mapping, ECC
realization/ # candidate generation and ranking
validation/  # hard constraints, NLI, numbers,
quality
detection/   # extraction, decoding, statistics
```

```
attacks/      # translation, paraphrase, summary,
splice
experiments/  # datasets, metrics, runners
```

A. Core Data Model

Listing 4. Illustrative Python data model.

```
from enum import Enum
from pydantic import BaseModel, Field

class Mutability(str, Enum):
    LOCKED = "locked"
    CONDITIONAL = "conditional"
    REALIZATION_ONLY = "realization_only"

class Event(BaseModel):
    id: str
    predicate: str
    roles: dict[str, str]
    polarity: str = "positive"
    modality: str = "asserted"
    time: dict = Field(default_factory=dict)
    quantity: dict | None = None
    attribution: str | None = None

class CarrierSite(BaseModel):
    carrier_id: str
    family: str
    target_node_ids: list[str]
    alternatives: list[str]
    recoverability: float
    quality_cost: float
```

B. Keyed Mapping Skeleton

Listing 5. Secret-keyed carrier assignment skeleton.

```
import hmac
from hashlib import sha256

def keyed_index(key, invariant_hash, nonce,
               carrier_id, symbol, option_count):
    msg = b"|".join([
        invariant_hash,
        nonce,
        carrier_id.encode(),
        symbol.to_bytes(4, "big")
    ])
    digest = hmac.new(key, msg, sha256).digest()
    return int.from_bytes(digest[:8], "big") % option_count
```

XIII. PROTOTYPE ROADMAP

A. Stage 1: Closed-Domain Proof of Concept

Begin with technical or news-style factual statements. Support entities, events, agent/patient roles, polarity, number, time, and causality. Start with carriers that are easy to validate: active/-passive voice, proposition order for independent facts, causal connective family, clause splitting/merging, and nominal/verbal realization.

B. Stage 2: Multilingual Parsing

Map English, Korean, and Hindi into one predicate and role inventory using a combination of multilingual structured extraction, dependency parsing, semantic-role labeling, named-entity normalization, numerical and temporal normalization, and multilingual embeddings as a fallback alignment mechanism.

TABLE IV

EVALUATION MATRIX: EACH RESEARCH QUESTION MAPPED TO ITS DECISIVE COMPARISON, MANIPULATED FACTOR, AND PRIMARY OUTCOME MEASURE.

RQ	Decisive comparison	Primary measure
RQ1	Truthprint vs. unconstrained paraphrase watermark	Invariant error rate (IER), contradiction rate
RQ2	Truthprint vs. token/embedding baselines, clean text	TPR@{1,0.1,0.01}%FPR, ROC-AUC
RQ3	Pre- vs. post-translation (1-hop, round-trip)	Payload BER, partial-recovery rate
RQ4	Sentence- vs. paragraph-level paraphrase	TPR@1%FPR vs. edit distance
RQ5	Watermarked vs. unwatermarked generation	Quality delta, added latency
RQ6	Recovery vs. document length	Authenticated bits recovered
RQ7	Adaptive attacker with vs. without invariant constraint	ValidRemoval rate (Eq. (1))

C. Stage 3: Document-Level Coding

Introduce multi-ary carrier alphabets, BCH or Reed–Solomon codes, carrier interleaving, synchronization markers, order-independent anchors, partial payload recovery, and provenance database lookup.

XIV. EXPERIMENTAL METHODOLOGY

A. Research Questions

We propose the following research questions:

- RQ1: Does Truthprint preserve immutable meaning better than unconstrained paraphrase-based watermarking?
- RQ2: What true-positive rate is achieved at fixed false-positive rates on clean text?
- RQ3: How much signal survives single-hop and round-trip translation?
- RQ4: How robust is Truthprint to sentence- and paragraph-level paraphrasing?
- RQ5: What quality and latency costs are introduced?
- RQ6: How many authenticated payload bits can be recovered as document length increases?
- RQ7: Can an adaptive attacker remove the watermark while preserving all locked invariants?

Table IV fixes, for each question, the single manipulated factor and the outcome that would confirm or falsify the corresponding hypothesis, so that results cannot be reported selectively.

B. Baselines

Experiments should include unwatermarked decoding, a representative Red–Green token watermark, SynthID-Text, SemStamp, SemaMark, semantic-invariant watermarking, SWAN, DEW, Truthprint without invariant validation, Truthprint without ECC, embedding-only Truthprint, and structure-only Truthprint. Standardized baseline implementations from an open watermarking toolkit should be used where available to ensure comparability [26].

TABLE V
CLEAN-TEXT DETECTION TEMPLATE.

Method	TPR@1%FPR	ROC-AUC	Quality
SynthID-Text	TBD	TBD	TBD
SemStamp	TBD	TBD	TBD
SWAN	TBD	TBD	TBD
Truthprint	TBD	TBD	TBD

C. Languages and Domains

Recommended languages include English, Korean, Hindi, Chinese, Arabic, and German or French. Domains should include factual news, encyclopedic explanations, technical documentation, scientific summaries, policy-style prose, dialogue, and creative writing. Code, formulas, legal clauses, and direct quotations should be reported separately due to low semantic carrier capacity.

D. Attacks

The evaluation should cover Unicode and punctuation edits, spelling correction, direct translation, round-trip and two-hop translation, lexical and syntactic paraphrasing, sentence splitting/merging, proposition reordering, deletion, insertion, splicing, summarization at multiple compression ratios, regeneration from bullet points, repeated detector-guided paraphrasing, and carrier-targeted rewriting.

E. Metrics

Detection metrics include ROC-AUC, true-positive rate at 1%, 0.1%, and 0.01% false-positive rates, false-negative rate, payload bit-error rate, complete and partial recovery rates, minimum reliable text length, and detector latency.

Semantic integrity metrics include entity consistency, role consistency, polarity accuracy, numerical accuracy, temporal consistency, causal-direction consistency, attribution consistency, bidirectional entailment, contradiction rate, and human factual-equivalence judgment.

A weighted invariant error rate can be defined as

$$\text{IER} = \frac{\sum_i w_i E_i}{\sum_i w_i}, \quad (20)$$

where critical categories such as polarity and numeric value receive larger weights or are treated as hard failures.

F. Result Tables

No unmeasured performance values are claimed in this paper. Tables V and VI are protocol templates that fix the reporting format for the multilingual evaluation; their cells are deliberately left unpopulated (TBD) because filling them requires the wide-coverage semantic frontend identified as future work.

TABLE VI
TRANSLATION-ROBUSTNESS TEMPLATE.

Method	EN→KO	EN→HI	KO→EN
SynthID-Text	TBD	TBD	TBD
SemStamp	TBD	TBD	TBD
DEW	TBD	TBD	TBD
Truthprint	TBD	TBD	TBD

XV. FALSIFIABLE HYPOTHESES

The following are hypotheses rather than results:

- H1: Truthprint will outperform token-level watermarking under translation because the carrier is reconstructed from language-independent semantics.
- H2: Explicit invariant validation will reduce errors in negation, quantity, time, causality, and attribution.
- H3: Truthprint may underperform token methods on very short text because semantic carrier capacity accumulates across sentences.
- H4: ECC and erasure-aware decoding will improve recovery under parsing errors and partial deletion.
- H5: Strong summarization will remain difficult because it removes propositions and their associated carriers.
- H6: Combining structured carriers with embedding-based fallback evidence will outperform either alone across diverse languages.

XVI. SECURITY AND DEPLOYMENT CONSIDERATIONS

Carrier mappings should depend on the secret key, invariant digest, document nonce, and carrier identity. This prevents simple global carrier rules and reduces forgery risk. Exact detector scores should be access controlled or coarsened to reduce oracle attacks. The detector should localize evidence by paragraph and avoid attributing a mixed document to one source.

Watermarks should support transparency rather than automated punishment. Every detection report should include the tested key and scheme version, text length, number of reliable carriers, calibrated false-positive rate, language and attack assumptions, confidence, and MAC verification status. User identifiers, raw prompts, and exact timestamps should not be embedded.

A. Deployment Path and Graceful Degradation

Truthprint is designed to slot into a serving stack as a semantic realization pass rather than as a modification of the token sampler. Two integration modes are practical. In the *inline* mode, the generator emits a draft, the encoder re-realizes admissible discourse units to carry payload symbols, and the validator gates each candidate; because the coding and cryptographic layers are independent of model size (Sec. IX), their overhead is bounded and does not grow with the LLM. In the *asynchronous* mode, watermarking runs as a post-generation service, which decouples watermark latency from the user-facing generation latency at the cost of a short delay before provenance is finalized.

Because capacity is unevenly distributed across text (Eq. 19), a deployable system must degrade gracefully. Low-capacity spans—code, formulas, legal clauses, and verbatim quotations—admit few invariant-preserving realizations and are therefore poor carriers. For such spans Truthprint records erasures and can optionally fall back to a token-level watermark such as SynthID-Text, yielding a layered scheme in which the semantic layer provides translation and paraphrase robustness while the token layer provides short-span coverage. The detector localizes evidence per paragraph, reports the number of reliable carriers alongside the calibrated false-positive rate, and abstains rather than guessing when the recovered capacity falls below the minimum reliable length. This makes the system’s competence explicit at detection time, which is the behavior an auditor or platform integrator requires.

XVII. LIMITATIONS

Multilingual semantic parsing remains imperfect. Not every proposition admits multiple safe realization alternatives. Highly constrained text has low carrier capacity. Some apparently stylistic alternations change emphasis or information structure. Translation can omit modality, evidentiality, subjects, or discourse relations. Strong summarization can destroy both content and watermark evidence. Finally, candidate generation and semantic validation add computational overhead compared with direct token-level methods.

XVIII. CONCLUSION

Truthprint is a language-independent, typed semantic IR for provenance watermarking. Its key contribution is a machine-checkable separation between immutable meaning and controlled realization choices. A secret-keyed encoder maps authenticated payload symbols to admissible realization classes, and a detector recovers those symbols after translation or paraphrasing through semantic parsing, canonicalization, error correction, and MAC verification.

The method cannot guarantee detection after arbitrary rewriting or semantic destruction. It instead targets a narrower and defensible objective: preserving provenance evidence across transformations that retain a substantial portion of the original meaning. Its value must be established through multilingual experiments, calibrated false-positive analysis, strong adaptive attacks, semantic-integrity evaluation, and direct comparison with token-, embedding-, and structure-based baselines.

REFERENCES

- [1] S. Dathathri *et al.*, “Scalable watermarking for identifying large language model outputs,” *Nature*, vol. 634, pp. 818–823, 2024. <https://www.nature.com/articles/s41586-024-08025-4>
- [2] I. Shumailov, Z. Shumaylov, Y. Zhao, N. Papernot, R. Anderson, and Y. Gal, “AI models collapse when trained on recursively generated data,” *Nature*, vol. 631, pp. 755–759, 2024. <https://www.nature.com/articles/s41586-024-07566-y>
- [3] European Parliament and Council, “Regulation (EU) 2024/1689 laying down harmonised rules on artificial intelligence (Artificial Intelligence Act),” *Official Journal of the European Union*, 2024. <https://eur-lex.europa.eu/eli/reg/2024/1689/oj>
- [4] A. Liu, L. Pan, Y. Lu, J. Li, X. Hu, X. Zhang, L. Wen, I. King, H. Xiong, and P. S. Yu, “A survey of text watermarking in the era of large language models,” *ACM Comput. Surv.*, vol. 57, no. 2, 2024. <https://dl.acm.org/doi/10.1145/3691626>
- [5] Google DeepMind, “SynthID-Text reference implementation,” GitHub repository, 2024. <https://github.com/google-deepmind/synthid-text>
- [6] Google, “SynthID: Tools for watermarking and detecting LLM-generated text,” Responsible GenAI Toolkit; describes the Transformers integration, accessed 2026. <https://ai.google.dev/responsible/docs/safeguards/synthid>
- [7] R. Omid, Y. Dong, and B. Wang, “On Google’s SynthID-Text LLM watermarking system: Theoretical analysis and empirical validation,” arXiv:2603.03410, 2026. <https://arxiv.org/abs/2603.03410>
- [8] X. Han, Q. Li, J. Ni, and M. Zulkernine, “Robustness assessment and enhancement of text watermarking for Google’s SynthID,” arXiv:2508.20228, 2025. <https://arxiv.org/abs/2508.20228>
- [9] J. Kirchenbauer, J. Geiping, Y. Wen, J. Katz, I. Miers, and T. Goldstein, “A watermark for large language models,” in *Proc. ICML*, 2023. <https://arxiv.org/abs/2301.10226>
- [10] X. Zhao, P. Ananth, L. Li, and Y.-X. Wang, “Provable robust watermarking for AI-generated text,” in *Proc. ICLR*, 2024. <https://arxiv.org/abs/2306.17439>
- [11] R. Kuditipudi, J. Thickstun, T. Hashimoto, and P. Liang, “Robust distortion-free watermarks for language models,” *Trans. Mach. Learn. Res.*, 2024. <https://arxiv.org/abs/2307.15593>
- [12] M. Christ, S. Gunn, and O. Zamir, “Undetectable watermarks for language models,” in *Proc. COLT*, 2024. <https://arxiv.org/abs/2306.09194>
- [13] P. Fernandez, A. Chaffin, K. Tit, V. Chappelier, and T. Furon, “Three bricks to consolidate watermarks for large language models,” in *Proc. IEEE WIFS*, 2023. <https://arxiv.org/abs/2308.00113>
- [14] A. Liu, L. Pan, X. Hu, S. Li, L. Wen, I. King, and P. S. Yu, “An unforgeable publicly verifiable watermark for large language models,” in *Proc. ICLR*, 2024. <https://arxiv.org/abs/2307.16230>
- [15] A. B. Hou *et al.*, “SemStamp: A semantic watermark with paraphrastic robustness for text generation,” in *Proc. NAACL*, 2024. <https://arxiv.org/abs/2310.03991>
- [16] J. Ren, H. Xu, Y. Liu, Y. Cui, S. Wang, D. Yin, and J. Tang, “A robust semantics-based watermark for large language models against paraphrasing,” in *Findings of NAACL*, 2024. <https://aclanthology.org/2024.findings-naacl/40/>
- [17] A. Liu, L. Pan, X. Hu, S. Meng, and L. Wen, “A semantic invariant robust watermark for large language models,” in *Proc. ICLR*, 2024. <https://arxiv.org/abs/2310.06356>
- [18] Z. Ye *et al.*, “SWAN: Semantic watermarking with Abstract Meaning Representation,” in *Proc. ACL*, 2026. <https://arxiv.org/abs/2605.04305>
- [19] J. Schäfer, C. Pilaszewicz, and G. Wunder, “Robust text watermarking for large language models via dual semantic embeddings,” arXiv:2606.31602, 2026. <https://arxiv.org/abs/2606.31602>
- [20] Z. He, B. Zhou, H. Hao, A. Liu, X. Wang, Z. Tu, Z. Zhang, and R. Wang, “Can watermarks survive translation? On the cross-lingual consistency of text watermark for large language models,” in *Proc. ACL*, 2024. <https://aclanthology.org/2024.acl-long.226/>
- [21] W. Qu, D. Yin, Z. He, W. Zou, T. Tao, J. Jia, and J. Zhang, “Provably robust multi-bit watermarking for AI-generated text via error correction code,” arXiv:2401.16820, 2024. <https://arxiv.org/abs/2401.16820>
- [22] T. Gloaguen, N. Jovanović, R. Staab, and M. Vechev, “Black-box detection of language model watermarks,” arXiv:2405.20777, 2024. <https://arxiv.org/abs/2405.20777>
- [23] T. Sander, P. Fernandez, A. Durmus, M. Douze, and T. Furon, “Watermarking makes language models radioactive,” in *Proc. NeurIPS*, 2024. <https://arxiv.org/abs/2402.14904>
- [24] X. Yi, S. Zheng, L. Wang, X. Wang, and L. He, “Unified attacks to large language model watermarks: Spoofing and scrubbing in unauthorized knowledge distillation,” arXiv:2504.17480, 2025. <https://arxiv.org/abs/2504.17480>
- [25] Y. Cheng, V. S. Sadasivan, M. Saberi, S. Saha, and S. Feizi, “Adversarial paraphrasing: A universal attack for humanizing AI-generated text,” arXiv:2506.07001, 2025. <https://arxiv.org/abs/2506.07001>
- [26] L. Pan *et al.*, “MarkLLM: An open-source toolkit for LLM watermarking,” in *Proc. EMNLP (System Demonstrations)*, 2024. <https://arxiv.org/abs/2405.10051>